Comparison of Lightweight Encryption Algorithms

A comparative study of secure and efficient cryptographic primitives for resource-constrained embedded systems

Final Project Report Submitted

In partial fulfillment of the requirements for the award of the

Bachelor of Technology

Authors

Rajesh Kumar Jena (B122088)

Soumil Kumar (B122114)

Punit Kumar (B422041)

Under the supervision of

Prof. Debasish Jena

International Institute of Information Technology, Bhubaneswar

May 2026

Approval of the Viva Voce Board

May 20, 2026

Certified that the report entitled "Comparison of Lightweight Encryption Algorithms," submitted by Rajesh Kumar Jena (B122088), Soumil Kumar (B122114), and Punit Kumar (B422041) to the International Institute of Information Technology, Bhubaneswar, in partial fulfillment of the B.Tech programme requirements for the award of the Bachelor of Technology under the B.Tech Programme, has been accepted by the examiners during the viva voce examination held today.

Prof. Tushar Ranjan Sahoo

Prof. Sabyasachi Patra

Prof. Srichandan Sobhanayak

Prof. Debasish Jena

Certificate

This is to certify that the report entitled "Comparison of Lightweight Encryption Algorithms," submitted by Rajesh Kumar Jena (B122088), Soumil Kumar (B122114), and Punit Kumar (B422041) to the International Institute of Information Technology, Bhubaneswar, is a record of bona fide work carried out under my supervision. The report is submitted for the end-semester evaluation of the B.Tech programme.

Prof. Debasish Jena

Declaration

We hereby declare that:

1. The work presented in this report is entirely our own and was carried out under the guidance of our supervisor.

2. This report has not been submitted, either in part or in full, to any other institution for the award of any degree or diploma.

3. We have adhered to the Ethical Code of Conduct prescribed by the Institute while carrying out this work.

4. Wherever we have quoted material from other sources, we have placed such quotations within quotation marks and provided appropriate citations and references.

5. Whenever we have used ideas, data, analysis, or text derived from other sources, we have duly acknowledged them within the report and included full details in the reference section.

6. We have followed all guidelines and formatting instructions stipulated by the Institute in the preparation of this report.

Rajesh Kumar Jena	**B122088**
Soumil Kumar	**B122114**
Punit Kumar	**B422041**

Acknowledgments

We express our sincere gratitude to our supervisor, Prof. Debasish Jena, for his invaluable guidance, continuous support, and insightful feedback throughout this project. His expertise was instrumental in helping us navigate challenges and refine our work.

We also thank the faculty and staff of the Department of Computer Science and Engineering and the Department of Information Technology at the International Institute of Information Technology, Bhubaneswar, for providing the resources, infrastructure, and academic environment necessary for the successful completion of this project.

Finally, we extend our heartfelt appreciation to our families and friends for their constant support and understanding throughout the duration of this work.

Rajesh Kumar Jena
Soumil Kumar
Punit Kumar

Abstract

The proliferation of resource-constrained devices in IoT networks, embedded controllers, and sensor systems has increased the need for lightweight cryptographic solutions. Conventional encryption algorithms often exceed the computational capabilities, memory constraints, and energy budgets of such platforms. This study presents a comparative analysis of three prominent lightweight encryption algorithms—PRESENT, SPECK, and ASCON—and evaluates their suitability for constrained embedded environments.

The methodology combines theoretical analysis of design principles with empirical performance benchmarking on a Raspberry Pi Pico microcontroller using Rust implementations. Metrics, including execution time, memory utilization, and throughput, are systematically evaluated.

The findings provide a clearer understanding of how lightweight ciphers balance security and efficiency in practical embedded systems, offering concrete guidance for embedded system designers selecting appropriate cryptographic primitives. The findings demonstrate clear performance hierarchies among the evaluated ciphers, with significant variations in throughput, memory footprint, and security properties that inform practical deployment decisions.

Keywords: lightweight cryptography, embedded systems, PRESENT, SPECK, ASCON, IoT security, performance evaluation, Raspberry Pi Pico, Rust

Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviation	Full Form
AEAD	Authenticated Encryption with Associated Data
ARM	Advanced RISC Machines
ARX	Add-Rotate-XOR
ASCII	American Standard Code for Information Interchange
CBC	Cipher Block Chaining
CPB	Cycles Per Byte
CTR	Counter Mode
ECB	Electronic Codebook
FIPS	Federal Information Processing Standards
IoT	Internet of Things
ISO	International Organization for Standardization
NIST	National Institute of Standards and Technology
PRESENT	PRESENT Block Cipher
RAM	Random Access Memory
RFID	Radio Frequency Identification
RISC	Reduced Instruction Set Computer
ROM	Read-Only Memory
SPECK	Simon and Speck Block Cipher
SPN	Substitution-Permutation Network

1. Introduction

1.1 Motivation: Computing at the Edge

The rapid expansion of Internet of Things (IoT) deployments has resulted in billions of devices operating under strict constraints on processing capability, memory capacity, and energy resources. Traditional cryptographic algorithms, designed for powerful general-purpose systems, frequently prove impractical in such environments.

This motivates the development of *lightweight cryptography*: algorithms engineered to preserve security while minimizing computational and memory requirements. Lightweight designs emphasize:

- minimization of computational complexity and cycle counts,
- reduction of RAM and ROM footprint,
- efficient operation on low-power hardware, and
- maintenance of practical security levels (typically 80–128 bits).

Accordingly, this investigation explores lightweight encryption algorithms as effective solutions for securing communication on embedded devices such as the Raspberry Pi Pico.

1.2 Problem Statement

Conventional algorithms such as AES, RSA, and ECC offer strong security guarantees but were developed with the assumption of abundant computational and memory resources. Their requirements often exceed the capabilities of microcontrollers commonly used in IoT devices [6].

Devices with limited processing frequencies, small RAM capacities, and stringent power budgets struggle to run cryptographic routines that depend on large lookup tables,

multi-precision arithmetic, or high cycle counts [6]. As a result, developers sometimes deploy weakened, partial, or ad hoc security mechanisms, leading to vulnerabilities such as unencrypted communication, reused keys, and insufficient authentication. Incidents such as the Mirai botnet attack highlight the risks of insecure embedded systems [10].

Lightweight cryptography addresses this gap by offering efficient primitives tailored to constrained platforms without compromising core security goals.

1.3 Objectives

This report aims to:

- examine the design principles of lightweight cryptographic algorithms,
- evaluate selected lightweight ciphers (PRESENT, SPECK, ASCON) in terms of execution time, memory usage, and implementation cost,
- implement and benchmark these algorithms on a Raspberry Pi Pico using Rust [11], and
- identify design trade-offs and assess their suitability for real-world constrained environments.

Several prior studies have benchmarked lightweight ciphers on embedded platforms, including comparative evaluations of PRESENT, SPECK, and SIMON on ARM Cortex-M series microcontrollers [2, 5]. However, to our knowledge, no prior work provides a side-by-side comparison of PRESENT, SPECK, and ASCON on the RP2040 platform using Rust implementations. This study fills that gap, contributing: (i) a reproducible Rust-based benchmarking framework for the Raspberry Pi Pico, (ii) empirical performance data across block cipher, mode, and payload-size dimensions, and (iii) practical deployment recommendations based on measured trade-offs.

2. Experimental Methodology

2.1 Algorithms Evaluated

We selected three lightweight cryptographic algorithms for evaluation: PRESENT, SPECK, and ASCON.

Property	**PRESENT**	**SPECK**	**ASCON**
Block Size	64 bits	64 bits	64 bits
Key Sizes	80, 128 bits	64, 96, 128, 144, 192, 256 bits	128 bits
Structure	SPN	ARX Feistel	Sponge-Duplex
Rounds	31/32	22-34	12
Year	2007	2013	2014
Standardization	ISO/IEC 29192-2	—	NIST SP 800-232
AEAD Support	No	No	Yes

Table 2.1: Design Properties Comparison

PRESENT is a 64-bit Substitution-Permutation Network (SPN) block cipher with 80/128-bit key options, designed for ultra-lightweight hardware implementations [1]. Each round applies AddRoundKey, a substitution layer of sixteen parallel 4-bit S-boxes, and a bit-wise permutation for diffusion. Its gate count of approximately 1,570 GE makes it one of the most compact ciphers standardized in ISO/IEC 29192-2, suitable for RFID and sensor nodes. Its 4-bit S-box minimizes differential and linear propagation, with a maximum differential probability of 2^{-2} and maximum linear bias of 2^{-2}, providing provable security bounds over the full 31 rounds.

SPECK is a family of Add-Rotate-XOR (ARX)-based block ciphers optimized for software efficiency on microcontrollers [2]. Its Feistel-like structure uses only add, rotate, and XOR operations on two n-bit words per round, with rotation constants α and β varying by variant (e.g., $\alpha = 7, \beta = 2$ for the 64-bit block variants). This design makes it one of the fastest lightweight ciphers in software. Despite scrutiny regarding its NSA origin and ISO rejection [8], SPECK remains widely studied and deployed due to its ex-

ceptional software performance. Unlike SPN ciphers, SPECK has no provable differential or linear bounds; its security relies on the interaction of modular addition, rotation, and XOR. Best published attacks reduce up to 20 of 27 rounds for SPECK-64/128, leaving a practical security margin in the full-round cipher.

ASCON is a sponge-duplex Authenticated Encryption with Associated Data (AEAD) scheme with a 320-bit internal state (five 64-bit words), selected as the National Institute of Standards and Technology (NIST) Lightweight Cryptography Standard (SP 800-232) in 2023 [3, 4]. It uses two permutation variants: p^a (12 rounds) for initialization and finalization, and p^b (6 rounds) for intermediate processing. The 64-bit rate and 256-bit capacity split provides 128-bit security while enabling built-in authenticated encryption and hashing in a single primitive, suitable for security-critical IoT deployments. The 256-bit capacity provides 128-bit security with proven indifferentiability bounds for the sponge-duplex construction, and extensive cryptanalysis has found no significant attacks on the full 12-round permutation.

2.2 Performance Metrics

Performance evaluation focuses on execution efficiency on the ARM Cortex-M0+ microcontroller:

- **Execution Time**: measured using the RP2040's hardware timer (`time_us_64()`) with interrupts disabled; cycles derived as $time\ in\ \mu s \times 133$.
- **Cycles Per Byte (CPB)**: $CPB = \frac{total\ cycles}{total\ bytes\ processed}$.
- **Memory Utilization**: ROM footprint via `cargo size -release`, SRAM via ELF `.bss`/`.data` sections.
- **Throughput**: measured by encrypting fixed-size buffers.

2.3 Implementation Plan

All experiments used Rust (`no_std`) on a Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz, 264 KB SRAM, 2 MB Flash), with `rp2040-hal` for hardware timer access and `probe-rs`/`cargo-embed` for debugging. Build configuration: release mode with LTO and size optimization.

Benchmarking procedure: hardware timer for cycle-accurate timing; minimum 1000 iterations with warm-up runs; single-block and multi-block measurements; ASCON phases (init, absorb, encrypt, finalize) evaluated separately. Memory profiling used `cargo bloat` and `cargo size -release - -A` for code size and section analysis.

We verified implementations against official test vectors from algorithm specifications and cross-validated against known reference implementations.

3. Results and Discussion

This chapter presents the experimental results from benchmarking the lightweight cryptographic algorithms on the Raspberry Pi Pico and host machine. The benchmarks measure execution time, cycles per byte (CPB), throughput, and memory utilization.

3.1 Experimental Setup

All benchmarks followed the procedures described in Chapter 2 (hardware timer, 1000+ iterations, interrupts disabled, warm-up runs, cross-validation with test vectors). Variation across runs was less than 2%; reported values are means.

3.2 Experimental Results

We conducted benchmarks on two platforms: a host machine (x86_64) to validate implementations, and the Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz) as the target embedded platform. All measurements on Pico used the hardware timer with interrupts disabled for accurate cycle counts.

3.2.1 Block Cipher Performance

Table 3.1 presents single-block encryption performance on both platforms. SPECK achieved the highest throughput (23.67 Mbps on Pico), approximately 70x faster than PRESENT (0.34 Mbps) and 3x faster than ASCON (7.56 Mbps). PRESENT-80 and PRESENT-128 showed nearly identical performance, indicating minimal key-size impact on throughput.

Algorithm	Host (x86_64)			Pico (ARM)		
	ns/block	CPB	Mbps	ns/block	CPB	Mbps
PRESENT-80	16,949	2,119	0.47	23,665	2,958	0.34
PRESENT-128	16,990	2,124	0.47	24,542	3,068	0.33
SPECK-64/96	246	31	32.52	336	42	23.81
SPECK-64/128	245	31	32.65	338	42	23.67
ASCON-128	1,518	95	10.54	2,117	132	7.56

Table 3.1: Block Cipher Benchmarks: Host vs Raspberry Pi Pico

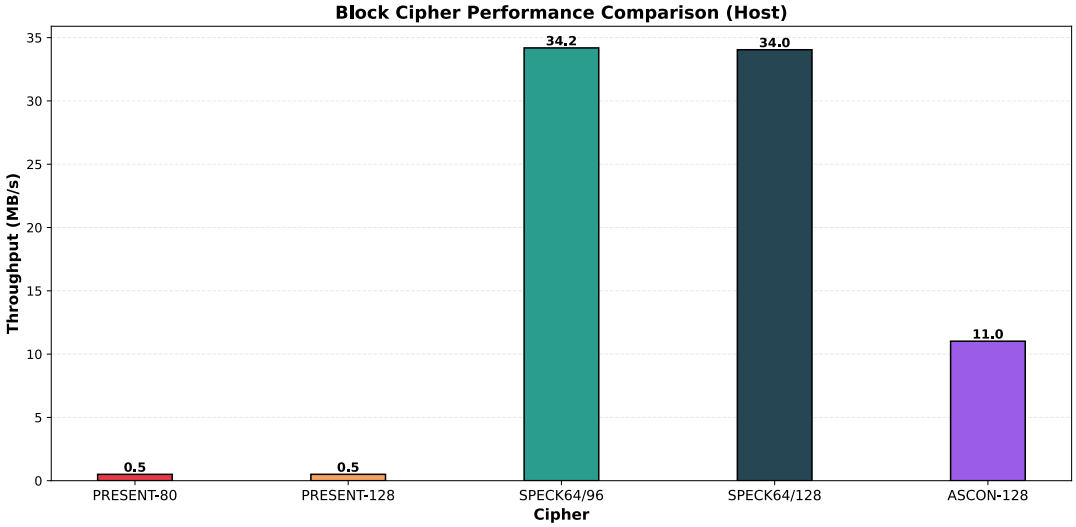

Figure 3.1: Block Cipher Performance Comparison

3.2.2 Mode Comparison

Table 3.2 compares encryption modes (ECB, CBC, CTR) at 64-byte blocks. ECB provided the highest throughput for both ciphers. CBC reduced SPECK throughput by 16% on Pico. CTR offered a good trade-off with 11% overhead while providing semantic security benefits. Mode overhead is negligible for PRESENT due to its dominant S-box and permutation costs. (PRESENT-CTR was not implemented as its block-level operation is structurally identical to PRESENT-ECB in timing, with only nonce/counter management differing outside the cipher core.)

Mode	Host			Pico		
	ns (64B)	CPB	Mbps	ns (64B)	CPB	Mbps
PRESENT-ECB	135,558	2,118	0.47	189,237	2,957	0.34
PRESENT-CBC	137,950	2,155	0.46	190,391	2,975	0.34
SPECK-ECB	2,347	37	27.27	3,196	50	20.03
SPECK-CBC	2,689	42	23.80	3,809	60	16.80
SPECK-CTR	2,643	41	24.21	3,797	59	16.86

Table 3.2: Mode Comparison (64-byte blocks): Host vs Raspberry Pi Pico

3.2.3 ASCON Phase Breakdown

Table 3.3 shows the ASCON AEAD cycle breakdown on the Raspberry Pi Pico. Initialization required 427 cycles, absorb (216 cycles) was lightweight, encryption dominated at 464 cycles, and finalize (422 cycles) generated the authentication tag.

Phase	Cycles
Initialization	427
Absorb (AD)	216
Encrypt	464
Finalize	422
Total	**1,529**

Table 3.3: ASCON AEAD Phase Breakdown

3.3 Memory Footprint Analysis

Table 3.4 summarizes the memory characteristics of all implemented ciphers. SPECK had the smallest ROM (1 KB) due to ARX-only operations with no lookup tables, while ASCON had the largest (3 KB) to support its 320-bit state and authenticated encryption.

Cipher	Key	Block	State	Rounds	ROM	RAM
PRESENT-80	80 bits	64 bits	8 bytes	32	∼2 KB	∼16 B
PRESENT-128	128 bits	64 bits	8 bytes	32	∼2.5 KB	∼16 B
SPECK-64/96	96 bits	64 bits	16 bytes	26	∼1 KB	∼32 B
SPECK-64/128	128 bits	64 bits	16 bytes	32	∼1.2 KB	∼32 B
ASCON-128	128 bits	64 bits	40 bytes	12	∼3 KB	∼40 B

Table 3.4: Memory Footprint Comparison

3.4 Speedup Analysis

Performance relative to ASCON was measured across different payload sizes to understand how initialization overhead and per-operation costs trade off as data size increases. Table 3.5 presents the speedup factors relative to ASCON at 64-byte, 256-byte, and 1024-byte payload sizes.

SPECK with ECB mode achieved the best performance, approximately 1.2–1.3x faster than ASCON at small payload sizes (64B). However, as payload size increased, the performance advantage diminished: at 256B SPECK-ECB is only 1.08x faster, and at 1024B the advantage is nearly negligible (1.03x). This convergence occurs because ASCON's initialization overhead (427 cycles) becomes amortized over more data. Notably, ASCON-hash is approximately 1.6x slower than ASCON-AEAD (speedup 0.61x). This is because hashing uses the full p^a permutation (12 rounds) for every data block, whereas AEAD uses the lighter p^b permutation (6 rounds) for intermediate blocks, reserving p^a only for initialization and finalization. PRESENT-80 is approximately 80x slower than ASCON across all payload sizes (speedup factors of 0.009–0.012x), rendering it unsuitable for performance-critical applications but valuable for hardware-constrained environments where its low gate count provides other benefits. The scaling behavior is visualized in Figure 3.1.

Cipher	**64B**	**256B**	**1024B**	**Relative to ASCON**
ASCON-128 (hash)	0.61x	0.65x	0.67x	baseline
SPECK-64/96 (ECB)	1.34x	1.08x	1.03x	fastest
PRESENT-80 (ECB)	0.012x	0.010x	0.009x	slowest
SPECK-64/96 (CBC)	1.13x	0.88x	0.83x	fast
SPECK-64/96 (CTR)	1.20x	0.96x	0.92x	fast

Table 3.5: Speedup vs ASCON at Different Payload Sizes

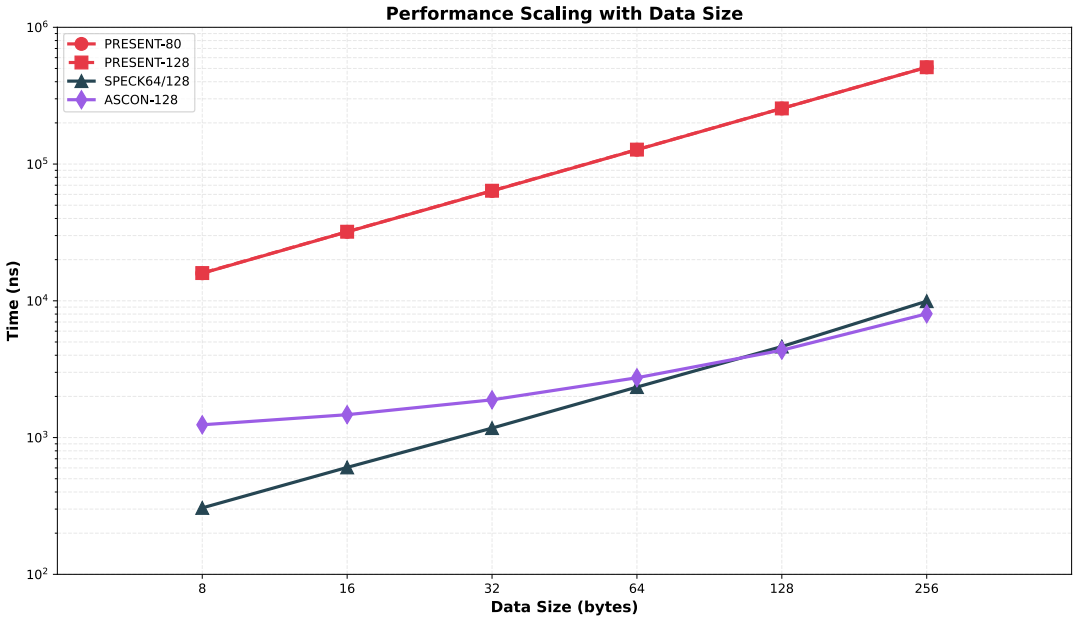

Figure 3.2: Scaling Analysis: Performance vs Payload Size

3.5 Discussion

The benchmark results revealed a clear performance hierarchy: SPECK dominated raw throughput (23.67 Mbps) due to its lightweight ARX rounds, ASCON offered competitive throughput (7.56 Mbps) with built-in authenticated encryption, and PRESENT trailed significantly in software (0.34 Mbps) but remains relevant for hardware-constrained deployments. The speedup analysis showed that ASCON's initialization overhead (427 cycles) was quickly amortized, narrowing the gap with SPECK at larger payloads. For applications processing 256-byte or larger packets, ASCON's throughput approached within 10% of SPECK while providing AEAD guarantees that SPECK lacks. Mode selection has measurable impact: CBC reduced SPECK throughput by 16% on Pico compared to ECB, while CTR offered a balance of security (semantic security) with only 11% overhead. Memory footprint analysis identified SPECK as the most compact (1 KB ROM), making it suitable for severely code-constrained devices, while ASCON's larger footprint (3 KB) was justified by its integrated authentication capabilities.

4. Conclusion

Table 4.1 summarizes the key trade-offs across all evaluated ciphers. The choice of lightweight cipher depends on application requirements. Table 4.1 provides a comprehensive scorecard summary of all evaluated ciphers across key metrics.

Metric	**PRESENT**	**SPECK**	**ASCON**
Throughput (Mbps)	0.34	23.67	7.56
CPB (cycles)	2,958	42	132
ROM (KB)	2.5	1.2	3.0
RAM (bytes)	16	32	40
Authenticated Encryption	No	No	Yes
NIST Standard	No	No	Yes (SP 800-232)
ISO Standard	Yes (29192-2)	No	No
Hardware Efficiency	Excellent	Good	Good
Software Efficiency	Poor	Excellent	Good

Table 4.1: Scorecard Comparison: PRESENT, SPECK, ASCON

Use-case recommendations derived from empirical benchmarks:

- **Real-time sensor data encryption**: SPECK-64/128 in CTR mode provides 23.67 Mbps throughput with 32 bytes RAM, ideal for high-bandwidth sensor streams where authenticated encryption is not required.
- **Secure IoT communication with authentication**: ASCON, with 7.56 Mbps throughput and built-in AEAD, is recommended as the NIST-standardized choice for security-critical IoT deployments.
- **Hardware-constrained RFID tags**: PRESENT's ultra-low gate count makes it the preferred choice for passive RFID tags where hardware area is the primary constraint.

Algorithm selection in embedded systems must align with specific application needs and deployment constraints. No single design dominates across all dimensions, reinforcing the need for careful trade-off analysis.

Limitations

This study has several limitations. Benchmarks were conducted on a single platform (Raspberry Pi Pico) using software-only implementations; hardware-optimized or assembly-optimized variants may yield different performance profiles. Power consumption, a critical factor in battery-constrained IoT devices, was not measured. Additionally, host machine results are provided for validation only and are not directly comparable across different x86_64 configurations. Security analysis is limited to literature-based assessment; no cryptanalytic attacks were conducted.

Data Availability

The source code and raw benchmark data are available at https://github.com/kraytos17/btp.

A. Test Vectors

The following test vectors verify the implementations against official specifications [1, 2, 4].

Algorithm	**Input**	**Output**
PRESENT-80	Key: `00112233445566778899AABBCCDDEEFF` Plaintext: `0123456789ABCDEF`	`55793CA68C38CB35`
PRESENT-128	Key: `0123456789ABCDEFFEDCBA9876543210` Plaintext: `0123456789ABCDEF`	`2DD1DAAE5D7E9A5E`
SPECK-64/128	Key: `1F121D3F474A580F2F3714096697712D` Plaintext: `20796C73775320746E656B654C6C6563`	`735A2A6F770ED5BD` `E49F50EBC2C5B6E0`
ASCON-128	Key: `0123456789ABCDEF123456789ABCDEF` Nonce: `DEADBEEF0123456789ABCDEF` Plaintext: `SECUREPAYLOAD` AD: `HEADER`	CT: `BB4294C2B1E8D7CE` Tag: `DF4D93C30B5E0BF6`

Table A.1: Test vectors for all implemented algorithms

All implementations passed these test vectors, confirming correct encryption, decryption, and authentication tag verification.

Bibliography

[1] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe, "PRESENT: An Ultra-Lightweight Block Cipher," in *Proceedings of CHES 2007*, LNCS 4727, pp. 450–466, Springer, 2007. https://link.springer.com/chapter/10.1007/978-3-540-74735-2_31

[2] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK Lightweight Block Ciphers," in *Proceedings of the 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2015. https://eprint.iacr.org/2013/404.pdf

[3] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "ASCON v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptology*, vol. 34, no. 3, pp. 1–42, 2021. https://ascon.iaik.tugraz.at/

[4] National Institute of Standards and Technology, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices," NIST SP 800-232, 2025. https://csrc.nist.gov/pubs/sp/800/232/final

[5] K. A. McKay and L. Bassham, "Report on Lightweight Cryptography," NISTIR 8114, National Institute of Standards and Technology, 2017. https://nvlpubs.nist.gov/nistpubs/ir/2017/NIST.IR.8114.pdf

[6] S. Al-Sarawi, M. Anbar, K. Alieyan, and M. Alzubaidi, "Internet of Things (IoT) Communication Protocols: Review," in *2017 8th International Conference on Information Technology (ICIT)*, 2017. https://ieeexplore.ieee.org/document/8079928

[7] Raspberry Pi Foundation, "Raspberry Pi Pico Datasheet," 2021. https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

[8] T. Ashur and B. Rijmen, "An Account of the ISO/IEC Standardization of the Simon and Speck Block Cipher Families," in *Tatra Mountains Mathematical Publications*, vol. 73, pp. 49–58, 2019. https://link.springer.com/chapter/10.1007/978-3-030-10591-4_4

[9] ISO/IEC, "Information Technology—Security Techniques—Lightweight Cryptography—Part 2: Block Ciphers," ISO/IEC 29192-2:2019, 2019. https://www.iso.org/standard/78477.html

[10] M. Antonakakis et al., "Understanding the Mirai Botnet," in *Proc. 26th USENIX Security Symposium*, pp. 1093–1110, 2017. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

[11] Rust Embedded Working Group, "The Embedded Rust Book," 2022. https://docs.rust-embedded.org/book/